# 03. AIR (Algebraic Intermediate Representation) 제약 조건

## 3.1 AIR 개념 이해

### 3.1.1 AIR이란?

AIR (Algebraic Intermediate Representation)은 계산의 정확성을 대수적 제약 조건으로 표현하는 방법입니다.

**핵심 아이디어:**

- 프로그램 실행을 다항식 방정식으로 표현
- 올바른 실행은 모든 제약 조건을 만족
- 잘못된 실행은 최소 하나의 제약 조건 위반

### 3.1.2 왜 AIR을 사용하는가?

1. **효율적 검증**: 복잡한 계산을 간단한 다항식 체크로 변환
2. **수학적 엄밀성**: 정확성을 수학적으로 증명 가능
3. **STARK 호환성**: STARK 증명 시스템과 자연스럽게 통합

## 3.2 VMAir 구조체

### 3.2.1 기본 정의

```
pub struct VMAir {}
```

```
impl<F: Field> BaseAir<F> for VMAir {
fn width(&self) -> usize {
11 // 추적 테이블의 컬럼 수
}
}
```

**width() 함수:**

- 추적 테이블의 컬럼 개수를 반환
- 이 구현에서는 11개 (스택 4개 + push_val 1개 + 플래그 5개 + extra_data 1개)

## 3.3 제약 조건 정의

### 3.3.1 Air trait 구현

```
impl<AB: AirBuilder> Air<AB> for VMAir {
fn eval(&self, builder: &mut AB) {
let main = builder.main();
let local = main.row_slice(0); // 현재 행
let next = main.row_slice(1); // 다음 행
// 제약 조건들...
}
}
```

**핵심 개념:**

- `local` : 현재 실행 단계의 상태 (row i)
- `next` : 다음 실행 단계의 상태 (row i+1)
- 제약 조건은 `local` 과 `next` 사이의 관계를 정의

### 3.3.2 추적 테이블 인덱스 참조

```
// local[i] 또는 next[i]로 접근
// 0-3: stack[0-3]
// 4: push_val
// 5: is_push
// 6: is_add
// 7: is_sub
// 8: is_mul
// 9: is_div
// 10: extra_data
```

## 3.4 제약 조건 상세 분석

### 3.4.1 초기 상태 제약 조건

```
for i in 0..9 {
builder.when_first_row().assert_zero(local[i]);
}
```

**의미:**

- 첫 번째 행(인덱스 0)에서 처음 9개 컬럼은 모두 0이어야 함
- 스택이 비어있고, 명령어 플래그도 모두 0인 초기 상태
- 인덱스 9, 10은 체크하지 않음 (is_div와 extra_data)

**수학적 표현:**

```
∀i ∈ [0, 8]: local[i] = 0 (첫 번째 행에서만)
```

### 3.4.2 Add 연산 제약 조건

```
// 제약 1: 결과 검증
builder
.when_transition()
.assert_zero(next[6] * (next[0] - local[0] - local[1]));

// 제약 2: 스택 이동 검증
builder
.when_transition()
.assert_zero(next[6] * (next[1] - local[2]));
```

**제약 1 분석:**

```
next[6] * (next[0] - local[0] - local[1]) = 0
```

- `next[6]` : Add 명령어 플래그 (1이면 Add 실행 중)

- `next[0]` : 다음 상태의 스택 최상위 값
- `local[0]` , `local[1]` : 현재 상태의 스택 상위 두 값

**동작 방식:**

- Add가 실행되지 않으면 ( `next[6] = 0` ): 제약 조건 자동 만족
- Add가 실행되면 ( `next[6] = 1` ): `next[0] = local[0] + local[1]` 이어야 함

**예시:**

```
현재 상태 (local): [20, 10, 5, 3, ...]

다음 상태 (next): [30, 5, 3, 0, ..., is_add=1]



검증: 1 * (30 - 20 - 10) = 1 * 0 = 0 ✓
```

**제약 2 분석:**

```
next[6] * (next[1] - local[2]) = 0
```

- Add 실행 시 `next[1] = local[2]`
- 스택의 세 번째 값이 두 번째 위치로 이동했는지 확인

**시각적 표현:**

```
Before Add: [a, b, c, d]

After Add: [a+b, c, d, 0]

↑ ↑

제약1 제약2
```

### 3.4.3 Sub 연산 제약 조건

```
builder
.when_transition()
.assert_zero(next[7] * (local[0] - local[1] - next[0]));
```

**수학적 표현:**

```
next[7] * (local[0] - local[1] - next[0]) = 0
```

**의미:**

- Sub 실행 시 ( `next[7] = 1` ): `next[0] = local[0] - local[1]`
- 첫 번째 값에서 두 번째 값을 뺀 결과

**예시:**

```
현재 상태 (local): [30, 10, 0, 0, ...]
다음 상태 (next): [20, 0, 0, 0, ..., is_sub=1]

검증: 1 * (30 - 10 - 20) = 1 * 0 = 0 ✓
```

**주의:** Add와 달리 스택 이동 제약이 별도로 없음 (공통 제약으로 처리)

### 3.4.4 Mul 연산 제약 조건

```
builder
.when_transition()
```

```
.assert_zero(next[8] * (local[0] * local[1] - next[0]));
```

**수학적 표현:**

```
next[8] * (local[0] * local[1] - next[0]) = 0
```

**의미:**

- Mul 실행 시 ( `next[8] = 1` ): `next[0] = local[0] * local[1]`
- 두 값의 곱이 결과

**예시:**

```
현재 상태 (local): [5, 4, 0, 0, ...]
다음 상태 (next): [20, 0, 0, 0, ..., is_mul=1]

검증: 1 * (5 * 4 - 20) = 1 * 0 = 0 ✓
```

### 3.4.5 Div 연산 제약 조건

```
builder
.when_transition()
.assert_zero(next[9] * (local[1] * next[0] + next[10] - local[0]));
```

**수학적 표현:**

```
next[9] * (local[1] * next[0] + next[10] - local[0]) = 0
```

**의미:**

- Div 실행 시 ( `next[9] = 1` ): `local[0] = local[1] * next[0] + next[10]`
- 나눗셈 검증: 피제수 = 제수 × 몫 + 나머지

**왜 이렇게 검증하는가?**

- 유한체에서 나눗셈은 역원 곱셈으로 정의됨
- 직접 나눗셈을 검증하기 어려움
- 대신 곱셈과 나머지로 역산하여 검증

**예시:**

```
현재 상태 (local): [23, 10, 0, 0, ...]

다음 상태 (next): [2, 0, 0, 0, ..., is_div=1, extra_data=3]


검증: 1 * (10 * 2 + 3 - 23) = 1 * 0 = 0 ✓

설명: 23 = 10 × 2 + 3 (23을 10으로 나누면 몫 2, 나머지 3)
```

**extra_data의 역할:**

- `next[10]` 에 나머지 저장
- 나눗셈의 정확성을 검증하는 데 필수적
- VM의 `perform_operation` 에서 계산됨

### 3.4.6 Push 연산 제약 조건

```
builder.when_transition().assert_zero(
```

```
next[5] * (
(next[0] - next[4]) + // 새 값이 스택 최상위에
(local[0] - next[1]) + // 이전 최상위가 두 번째로
(local[1] - next[2]) + // 이전 두 번째가 세 번째로
(local[2] - next[3]) // 이전 세 번째가 네 번째로
)
);
```

**수학적 표현:**

```
next[5] * (
(next[0] - next[4]) +
(local[0] - next[1]) +
(local[1] - next[2]) +
(local[2] - next[3])
) = 0
```

**의미:**

- Push 실행 시 ( `next[5] = 1` ): 모든 괄호 안의 값이 0이어야 함
- `next[0] = next[4]` : push_val이 스택 최상위로
- `next[1] = local[0]` : 이전 값들이 한 칸씩 뒤로 이동

**예시:**

```
현재 상태 (local): [10, 20, 30, 0, ...]
다음 상태 (next): [5, 10, 20, 30, push_val=5, is_push=1, ...]
```

```
검증:

(5 - 5) = 0 ✓

(10 - 10) = 0 ✓

(20 - 20) = 0 ✓

(30 - 30) = 0 ✓

합: 0 + 0 + 0 + 0 = 0 ✓
```

### 3.4.7 공통 스택 이동 제약 조건

```
builder

.when_transition()

.assert_zero((next[6] + next[7] + next[8] + next[9]) * (next[1] - local[2]));
```

**수학적 표현:**

```
(next[6] + next[7] + next[8] + next[9]) * (next[1] - local[2]) = 0
```

**의미:**

- 이항 연산(Add, Sub, Mul, Div) 실행 시
- 스택의 세 번째 값이 두 번째 위치로 이동해야 함

**왜 필요한가?**

- 모든 이항 연산은 스택 상위 두 값을 소비
- 나머지 값들이 앞으로 이동

- 이 제약은 그 이동을 검증

**예시:**

```
현재 상태 (local): [a, b, c, d, ...]

Add/Sub/Mul/Div 후: [result, c, d, 0, ...]

↑ ↑

제약 검증
```

## 3.5 제약 조건의 수학적 의미

### 3.5.1 조건부 제약 (Conditional Constraints)

모든 제약 조건은 다음 형태를 가집니다:

```
flag * (constraint_expression) = 0
```

**두 가지 경우:**

1. `flag = 0` : 제약 조건이 비활성화 (자동으로 만족)
2. `flag = 1` : `constraint_expression = 0` 이어야 함

### 3.5.2 원-핫 인코딩 보장

명령어 플래그들 ( `is_push` , `is_add` , 등)은 원-핫 인코딩되어야 합니다:

- 정확히 하나만 1
- 나머지는 모두 0

**이는 어떻게 보장되는가?**

- VM의 `get_trace` 함수에서 보장
- 각 행은 정확히 하나의 명령어만 나타냄
- AIR은 이를 가정하고 제약 조건 정의

### 3.5.3 상태 전이 그래프

```
State i (local) ──[instruction]──> State i+1 (next)

↓ ↓

[a,b,c,d] ──[Add, flag=1]──> [a+b,c,d,0]

↓ ↓

제약 조건으로 검증: next[0] = local[0] + local[1]
```

## 3.6 제약 조건 검증 과정

### 3.6.1 STARK 증명에서의 역할

1. **추적 생성**: VM이 실행 추적 생성
2. **다항식 변환**: 추적을 다항식으로 인코딩
3. **제약 평가**: 모든 행에서 제약 조건 평가
4. **증명 생성**: 제약이 만족됨을 증명

### 3.6.2 검증자의 관점

검증자는 다음을 확인합니다:

```
∀ row i in trace:
```

```
- 초기 상태 제약 (i = 0일 때)
- 상태 전이 제약 (i < len-1일 때)
- 각 명령어별 제약
```

### 3.6.3 제약 위반 예시

**잘못된 Add 연산:**

```
현재 상태: [10, 20, 0, 0, ...]
다음 상태: [40, 0, 0, 0, ..., is_add=1] // 잘못됨! 10+20=30이어야 함

제약 검증: 1 * (40 - 10 - 20) = 1 * 10 = 10 ≠ 0 ✗
```

이 경우 제약 조건이 위반되어 증명 생성 실패!

## 3.7 AIR 설계 원칙

### 3.7.1 완전성 (Completeness)

- 올바른 실행은 항상 모든 제약 조건을 만족
- 정직한 증명자는 항상 증명 생성 가능

### 3.7.2 건전성 (Soundness)

- 잘못된 실행은 최소 하나의 제약 조건 위반
- 부정직한 증명자는 증명 생성 불가능

### 3.7.3 효율성

- 제약 조건은 간단한 다항식
- 평가 비용이 낮음
- 대규모 계산에도 확장 가능

## 3.8 실제 제약 조건 적용 예시

### 3.8.1 전체 프로그램 검증

프로그램:

```
Push(10)
Push(20)
Add
```

추적 테이블:

```
Row | s[0] | s[1] | s[2] | s[3] | pval | isPush | isAdd | ... | extra
----|------|------|------|------|------|--------|-------|-----|------
0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 0
1 | 10 | 0 | 0 | 0 | 10 | 1 | 0 | ... | 0
2 | 20 | 10 | 0 | 0 | 20 | 1 | 0 | ... | 0
3 | 30 | 0 | 0 | 0 | 0 | 0 | 1 | ... | 0
```

**Row 0 → Row 1 (Push 10):**

```
next[5] * ((10-10) + (0-0) + (0-0) + (0-0)) = 1 * 0 = 0 ✓
```

**Row 1 → Row 2 (Push 20):**

```
next[5] * ((20-20) + (10-10) + (0-0) + (0-0)) = 1 * 0 = 0 ✓
```

**Row 2 → Row 3 (Add):**

```
next[6] * (30 - 20 - 10) = 1 * 0 = 0 ✓

next[6] * (0 - 0) = 1 * 0 = 0 ✓
```

모든 제약 조건이 만족되므로 증명 생성 가능!

## 3.9 요약

AIR 제약 조건은 zkVM의 핵심입니다:

- 프로그램 실행의 정확성을 수학적으로 정의
- 각 명령어의 의미를 다항식 방정식으로 표현
- STARK 증명 시스템의 기반 제공

다음 문서에서는 이 제약 조건을 사용하여 실제로 어떻게 증명을 생성하는지 살펴보겠습니다.